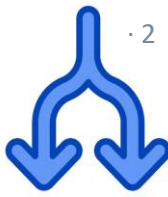


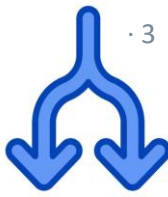
Practical Concurrent and Parallel Programming V

Testing & Verification

Raúl Pardo



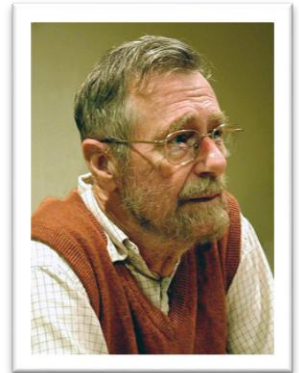
- Assignment 1 grades
 - Make sure that you get the correct grade in LearnIT
 - We will do our best, but errors may occur
 - At the end of the course, we check eligibility for the exam by looking at your grades in LearnIT



- Definitions of thread-safety
 - Classes
 - Programs
- Safe publication
- Immutability
- Synchronization primitives
 - Semaphores
 - Barriers
- Producer-consumer problem
- Instance confinement

*“Program **testing** can be used to **show the presence of bugs**, but
never to show their absence!”*

Edsger W. Dijkstra



- Intro to concurrency properties
- Testing
 - Intro to JUnit
 - Counter
 - Bounded Buffer
 - Deadlocks
- BONUS: Formal Verification
 - Java Path Finder





*A (concurrent) program is correct if
it satisfies its specification*



*A (concurrent) program is correct if
it satisfies its specification*

Specifications are often stated as a
collection of program *properties*



*A (concurrent) program is correct if
it satisfies its specification*

Specifications are often stated as a
collection of program *properties*

A property can be seen as a single
statement of a program specification

Concurrency properties (intuition)



- Traditionally, properties of concurrent programs are split into:
 - Safety (intuition) – *“Something bad never happens”*

Ex. 1: Two intersection traffic lights are never green at the same time

Ex. 2: The field size of a collection is never less than 0

- Liveness (intuition) – *“Something good will eventually happen”*

Ex. 1: The traffic light will eventually switch to red

Ex. 2: It should always be possible to eventually add elements to the collection

Interleavings

```
// shared variable
int counter = 0;

// two threads
for(int i=0; i<2; i++){
    new Thread(() -> {
        while(true) {
            int temp = counter;    (1)
            temp = counter + 1;    (2)
            counter = temp;        (3)
        }
    }).start();
}
```

Assuming that (1), (2) and (3) are atomic.



Some interleavings are

1. **t1(1)**, **t1(2)**, **t1(3)**, t2(1), t2(2), t2(3),...
2. t2(1), t2(2), t2(3), **t1(1)**, **t1(2)**, **t1(3)**,...
3. **t1(1)**, **t1(2)**, **t1(3)**, **t1(1)**, **t1(2)**, **t1(3)**,...
4. t2(1), t2(2), t2(3), t2(1), t2(2), t2(3),...

Interleavings

```
// shared variable
int counter = 0;

// two threads
for(int i=0; i<2; i++){
    new Thread(() -> {
        while(true) {
            int temp = counter;    (1)
            temp = counter + 1;    (2)
            counter = temp;        (3)
        }
    }).start();
}
```

Assuming that (1), (2) and (3) are atomic.



Some interleavings are

1. **t1(1)**, **t1(2)**, **t1(3)**, t2(1), t2(2), t2(3),...
2. t2(1), t2(2), t2(3), **t1(1)**, **t1(2)**, **t1(3)**,...
3. **t1(1)**, **t1(2)**, **t1(3)**, **t1(1)**, **t1(2)**, **t1(3)**,...
4. t2(1), t2(2), t2(3), t2(1), t2(2), t2(3),...

But we also have

1. **t1(1)**, t2(1), **t1(2)**, t2(2), **t1(3)**, t2(3),...
2. **t1(1)**, **t1(2)**, t2(1), t2(2), **t1(3)**, t2(3),...

These produce race conditions

Testing concurrent programs



- Testing concurrent programs is about writing tests to find undesired interleavings (if any)
 - These are commonly known as *counterexamples*
 - They show an interleaving that violates a property
- Since concurrent execution is non-deterministic, it is not guaranteed that tests will trigger undesired interleavings
- Today we will see strategies to design tests to find interleavings that violate a property

But we also have

1. $t1(1), t2(1), t1(2), t2(2), t1(3), t2(3), \dots$
2. $t1(1), t2(2), t2(1), t2(2), t1(3), t2(3), \dots$

These produce race conditions

Counterexamples define the type of property

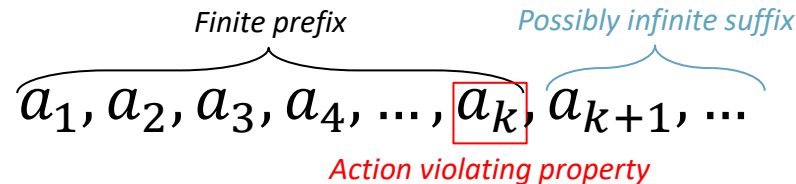


Safety and liveness are precisely defined by the type of counterexample that invalidate them

- We say that a property is a safety property if a counterexample is a finite prefix of a (possibly infinite) interleaving where the property does not hold
- We say that a property is a liveness property if a counterexample is an infinite interleaving where the property never holds

This is the definition of the properties.
Slide 7 only describes intuition.

- Safety property
 - A counterexample is a finite prefix of a (possibly infinite) interleaving where the property does not hold



- Ex. 1: Two intersection traffic lights are never green at the same time
 - *Counterexample*: a finite interleaving (finite sequence of traffic light states) where the two traffic lights are green at the same time.
- Ex. 2: The field size of a collection is never less than 0
 - *Counterexample*:

Counterexamples in safety properties



menti.com
2238 9090

- Safety property
 - A counterexample is a *finite prefix* of a (possibly infinite) interleaving where the property does not hold

$$\underbrace{a_1, a_2, a_3, a_4, \dots, a_k}_{\text{Finite prefix}} \underbrace{a_{k+1}, \dots}_{\text{Possibly infinite suffix}}$$

a_k
Action violating property

- Ex. 1: Two intersection traffic lights are never green at the same time
 - *Counterexample*: a finite interleaving (finite sequence of traffic light states) where the two traffic lights are green at the same time.
- Ex. 2: The field size of a collection is never less than 0
 - *Counterexample*: Can you give a counterexample for this property?



- Liveness property
 - A counterexample is an *infinite* interleaving where the property never holds.
- Ex. 1: The traffic light will eventually switch to red
 - *Counterexample*: an infinite interleaving (infinite sequence of traffic light states) where the traffic light is never red. For instance, a traffic light that is always green.

$a_1, a_2, \dots$ and $a_i = \text{"switch light to green"}$

- Ex. 2: It should always be possible to eventually add elements to the collection
 - *Counterexample*: an infinite interleaving when a thread can never add an element to the collection. For instance, in a system with 3 writers, writers 1 and 3 write infinitely often and writer 2 can never write

$w1(\text{writes}), w2(\text{enqueued}), [w3(\text{enqueued}), w1(\text{stop_write}), w3(\text{writes}),$
 $w1(\text{enqueued}), w3(\text{stop_write}), w1(\text{writes})]^*$

- Liveness property
 - A counterexample is an *infinite* interleaving where the property never holds.
- Ex. 1: The traffic light will eventually switch to red
 - *Counterexample*: an infinite interleaving (infinite sequence of traffic light states) where the traffic light is never red. For instance, a traffic light that is always green.

$a_1, a_2, \dots$ and $a_i = \text{"switch light to green"}$

- Ex. 2: It should always be possible to eventually add elements to the collection
 - *Counterexample*: an infinite interleaving when a thread can never add an element to the collection. For instance, in a system with 3 writers, writers 1 and 3 write infinitely often and writer 2 can never write

w1(writes), w2(enqueued), [w3(enqueued), w1(stop_write), w3(writes),
w1(enqueued), w3(stop_write), w1(writes)]*

Infinite suffix of the interleaving

Types of concurrency properties



- **Safety**
- Liveness

Today we focus only on
safety properties

Testing Concurrent Programs (Counter)

- **Functional Correctness tests**
 - These tests focus on testing that program behaves (functions) correctly when executed concurrently
 - For instance, data structures
 - *This lecture focuses on this type of tests*
- Performance tests (in lecture 8 with Peter)
 - These tests focus on measuring the execution time of concurrent programs
 - We will see a more accurate (and statistically stronger) method to measure performance than the method in the book

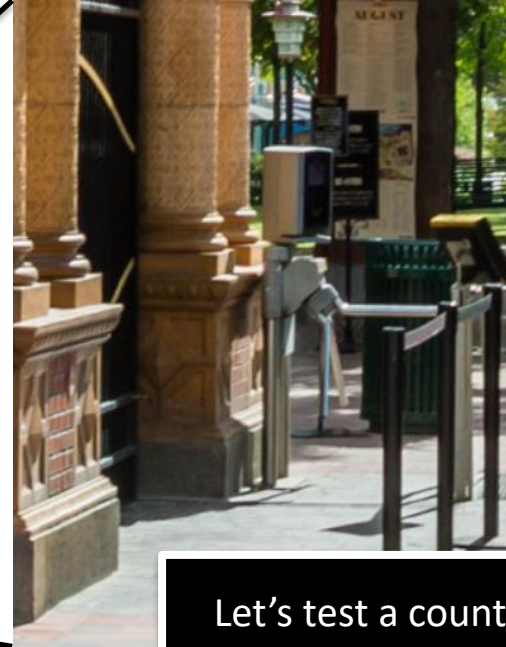
- JUnit is a popular unit test framework for Java programs
- It makes it easy to implement and run tests
- Some useful features are:
 - Execute initialization tasks
 - Running tests repeatedly
 - Define and automatically execute sets of input parameters for a test
 - Compatibility with build tools, such as Gradle
 - ...



Threads in Java – Example

Tivoli entrance turnstile

Déjà vu from lecture 1



Let's test a counter class that counts the number of visitors that cross the turnstile

Sequential tests in JUnit

· 21



Class to test

```
class CounterDR implements Counter {  
  
    private int count;  
  
    public CounterDR() {  
        count = 0;  
    }  
  
    public void inc() {  
        count++;  
    }  
  
    public int get() {  
        return count;  
    }  
}
```

Test Class

```
// several imports  
  
public class CounterTest {  
  
    private Counter count;  
  
    @BeforeEach  
    public void initialize() {  
        count = new CounterDR();  
    }  
  
    @RepeatedTest(10)  
    @DisplayName("Counter Sequential")  
    public void testingCounterSequential()  
    {  
        int localSum = 0;  
        for (int i = 0; i < 10_000; i++) {  
            count.inc();  
            localSum++;  
        }  
        assertTrue(count.get() == localSum);  
    }  
    ...  
    // other tests  
}
```

Sequential tests in JUnit

· 21



Class to test

```
class CounterDR implements Counter {  
  
    private int count;  
  
    public CounterDR() {  
        count = 0;  
    }  
  
    public void inc() {  
        count++;  
    }  
  
    public int get() {  
        return count;  
    }  
}
```

Counter variable that will
be used in the tests

Test Class

```
// several imports  
  
public class CounterTest {  
  
    private Counter count;  
  
    @BeforeEach  
    public void initialize() {  
        count = new CounterDR();  
    }  
  
    @RepeatedTest(10)  
    @DisplayName("Counter Sequential")  
    public void testingCounterSequential()  
    {  
        int localSum = 0;  
        for (int i = 0; i < 10_000; i++) {  
            count.inc();  
            localSum++;  
        }  
        assertTrue(count.get() == localSum);  
    }  
    ...  
    // other tests  
}
```

Sequential tests in JUnit

· 21



Class to test

```
class CounterDR implements Counter {  
  
    private int count;  
  
    public CounterDR() {  
        count = 0;  
    }  
  
    public void inc() {  
        count++;  
    }  
  
    public int get() {  
        return count;  
    }  
}
```

Counter variable that will
be used in the tests

This method is executed
before each test. It is
useful to initialize the
objects to test

Test Class

```
// several imports  
  
public class CounterTest {  
  
    private Counter count;  
  
    @BeforeEach  
    public void initialize() {  
        count = new CounterDR();  
    }  
  
    @RepeatedTest(10)  
    @DisplayName("Counter Sequential")  
    public void testingCounterSequential()  
    {  
        int localSum = 0;  
        for (int i = 0; i < 10_000; i++) {  
            count.inc();  
            localSum++;  
        }  
        assertTrue(count.get() == localSum);  
    }  
    ...  
    // other tests  
}
```

Sequential tests in JUnit

· 21



Class to test

```
class CounterDR implements Counter {
```

```
    private int count;
```

```
    public CounterDR() {  
        count = 0;  
    }
```

```
    public void inc() {  
        count++;  
    }
```

```
    pu  
    }  
}
```

Counter variable that will be used in the tests

This method is executed before each test. It is useful to initialize the objects to test

First, we define the type of test. One might use `@Test` (regular test), `@RepeatedTest(X)` the test is executed X times, or `@ParameterizedTest` with an input generator (see next slides)

Test Class

```
// several imports
```

```
public class CounterTest {
```

```
    private Counter count;
```

```
    @BeforeEach
```

```
    public void initialize() {  
        count = new CounterDR();  
    }
```

```
    @RepeatedTest(10)
```

```
    @DisplayName("Counter Sequential")
```

```
    public void testingCounterSequential()  
    {
```

```
        int localSum = 0;
```

```
        for (int i = 0; i < 10_000; i++) {  
            count.inc();  
            localSum++;  
        }
```

```
        assertTrue(count.get() == localSum);  
    }
```

```
    ...
```

```
    // other tests
```

```
}
```

Sequential tests in JUnit

· 21



Class to test

```
class CounterDR implements Counter {
```

```
    private int count;
```

```
    public CounterDR() {  
        count = 0;  
    }
```

```
    public void inc() {  
        count++;  
    }
```

```
    pu  
    }  
}
```

Counter variable that will be used in the tests

This method is executed before each test. It is useful to initialize the objects to test

First, we define the type of test. One might use `@Test` (regular test), `@RepeatedTest(X)` the test is executed X times, or `@ParameterizedTest` with an input generator (see next slides)

Test Class

```
// several imports
```

```
public class CounterTest {
```

```
    private Counter count;
```

```
    @BeforeEach
```

```
    public void initialize() {  
        count = new CounterDR();  
    }
```

```
    @RepeatedTest(10)
```

```
    @DisplayName("Counter Sequential")
```

```
    public void testingCounterSequential()  
    {
```

```
        int localSum = 0;
```

```
        for (int i = 0; i < 10_000; i++) {  
            count.inc();  
            localSum++;  
        }
```

```
        assertTrue(count.get() == localSum);  
    }
```

```
    ...
```

```
    // other tests
```

```
}
```

Some text to display when printing the result of the test

Sequential tests in JUnit

· 21



Class to test

```
class CounterDR implements Counter {
```

```
    private int count;
```

```
    public CounterDR() {  
        count = 0;  
    }
```

```
    public void inc() {  
        count++;  
    }
```

```
    pu  
}  
}
```

Counter variable that will be used in the tests

This method is executed before each test. It is useful to initialize the objects to test

First, we define the type of test. One might use `@Test` (regular test), `@RepeatedTest(X)` the test is executed X times, or `@ParameterizedTest` with an input generator (see next slides)

Body of the test. In this case we execute `inc()` 10000 times

Test Class

```
// several imports
```

```
public class CounterTest {
```

```
    private Counter count;
```

```
    @BeforeEach  
    public void initialize() {  
        count = new CounterDR();  
    }
```

```
    @RepeatedTest(10)  
    @DisplayName("Counter Sequential")  
    public void testingCounterSequential()  
    {  
        int localSum = 0;  
        for (int i = 0; i < 10_000; i++) {  
            count.inc();  
            localSum++;  
        }  
        assertTrue(count.get() == localSum);  
    }  
    ...  
    // other tests  
}
```

Some text to display when printing the result of the test

Sequential tests in JUnit

· 22



Class to test

```
class CounterDR implements Counter {  
  
    private int count;  
  
    public CounterDR() {  
        count = 0;  
    }  
  
    public void inc() {  
        count++;  
    }  
  
    public int get() {  
        return count;  
    }  
}
```

The test finishes with some assertions.
Here we check that the final value of
count equals our local sum.

You may also add assertions during the
execution of the test.

Test Class

```
// several imports  
  
public class CounterTest {  
  
    private Counter count;  
  
    @BeforeEach  
    public void initialize() {  
        count = new CounterDR();  
    }  
  
    @RepeatedTest(10)  
    @DisplayName("Counter Sequential")  
    public void testingCounterSequential()  
    {  
        int localSum = 0;  
        for (int i = 0; i < 10_000; i++) {  
            count.inc();  
            localSum++;  
        }  
        assertTrue(count.get() == localSum);  
    }  
    ...  
    // other tests  
}
```

Sequential tests in JUnit

· 23



Class to test

```
class CounterDR implements Counter {  
  
    private int count;  
  
    public CounterDR() {  
        count = 0;  
    }  
  
    public void inc() {  
        count++;  
    }  
}
```

To run tests in gradle we use:

```
$ gradle cleanTest test --tests <package>.<test_class>
```

In this example,

```
$ gradle cleanTest test --tests lecture05.CounterTest
```

(`cleanTest` ensures a fresh environment for running the test, it is not always necessary)

Test Class

```
// several imports  
  
public class CounterTest {  
  
    private Counter count;  
  
    @BeforeEach  
    public void initialize() {  
        count = new CounterDR();  
    }  
  
    @RepeatedTest(10)  
    @DisplayName("Counter Sequential")  
    public void testingCounterSequential()  
    {  
        int localSum = 0;  
        for (int i = 0; i < 10_000; i++) {  
            count.inc();  
            localSum++;  
        }  
        assertTrue(count.get() == localSum);  
    }  
    ...  
    // other tests  
}
```

Concurrent Correctness Test – Counter

· 24



- Now we extend the test to multiple threads (or turnstiles)



You must take into account the following points when developing a test for a concurrent program:

1. Precisely define the property you want to test
2. If you are going to test multiple implementations, it is useful to define an *interface* for the class you are testing
3. Concurrent tests require a setup for starting and running multiple threads
 - Maximize contention to avoid a sequential execution of the threads
 - You may need to define thread classes
4. Run the tests multiple times and with different setups to try to maximize the number of interleavings tested

Important
for testing
any kind of
programs

Crucial for
testing
concurrent
programs

- Precisely define the property you want to test
 - Use assertions to test properties
- *“after N threads execute $inc()$ X times, the value of the counter must be equal to $N * X$ ”*

```
Class CounterTest {  
  
    Counter count;  
    ...  
    public void testingCounterParallel(int nrThreads, int N) {  
        // body of the test  
        assert(N*nrThreads == count.get());  
    }  
    ...  
}
```

Concurrent Correctness Test – Counter

· 26

- Precisely define the property you want to test
 - Use assertions to test properties
- *“after N threads execute $inc()$ X times, the value of the counter must be equal to $N * X$ ”*



menti.com
2238 9090

```
Class CounterTest {  
  
    Counter count;  
    ...  
    public void testingCounterParallel(int nrThreads, int N) {  
        // body of the test  
        assert(N*nrThreads == count.get());  
    }  
    ...  
}
```

Is this a safety or liveness property?



- If you are going to test multiple implementations, it is useful to define an *interface* for the class you are testing

```
public interface Counter {  
    public void inc();  
    public int get();  
}
```

```
class CounterDR implements Counter {  
  
    private int count;  
  
    public CounterDR() {  
        count = 0;  
    }  
  
    public void inc() {  
        count++;  
    }  
  
    public int get() {  
        return count;  
    }  
}
```

```
class CounterSync implements Counter {  
  
    private int count;  
  
    public CounterSync() {  
        count = 0;  
    }  
  
    public synchronized void inc() {  
        count++;  
    }  
  
    public int get() {  
        return count;  
    }  
}
```

```
class CounterAto implements Counter {  
  
    private AtomicInteger count;  
  
    public CounterAto() {  
        count = new AtomicInteger(0);  
    }  
  
    public void inc() {  
        count.incrementAndGet();  
    }  
  
    public int get() {  
        return count.get();  
    }  
}
```



- If you are going to test multiple implementations, it is useful to define an *interface* for the class you are testing

```
public interface Counter {  
    public void inc();  
    public int get();  
}
```

A thread-safe integer class, with methods to increase, decrease, etc. the integer

```
class CounterDR implements Counter {  
  
    private int count;  
  
    public CounterDR() {  
        count = 0;  
    }  
  
    public void inc() {  
        count++;  
    }  
  
    public int get() {  
        return count;  
    }  
}
```

```
class CounterSync implements Counter {  
  
    private int count;  
  
    public CounterSync() {  
        count = 0;  
    }  
  
    public synchronized void inc() {  
        count++;  
    }  
  
    public int get() {  
        return count;  
    }  
}
```

```
class CounterAto implements Counter {  
  
    private AtomicInteger count;  
  
    public CounterAto() {  
        count = new AtomicInteger(0);  
    }  
  
    public void inc() {  
        count.incrementAndGet();  
    }  
  
    public int get() {  
        return count.get();  
    }  
}
```



- Maximize thread contention
 - Maximizing the number of threads running concurrently
- A cyclic barrier may be used to decrease the chance that threads are executed sequentially

```
class TestCounter {  
  
    // Shared variable for the tests  
    CyclicBarrier barrier;  
  
    ...  
  
    public void testingCounterParallel(int nrThreads, int N) {  
        ...  
  
        // init barrier  
        barrier = new CyclicBarrier(nrThreads + 1);  
  
        for (int i = 0; i < nrThreads; i++) {  
            new Thread(() -> {  
                barrier.await(); // wait until all threads are ready  
                // thread execution  
                barrier.await(); // wait until all threads are finished  
            }).start();  
        }  
  
        try {  
            barrier.await();  
            barrier.await();  
        } catch (InterruptedException | BrokenBarrierException e) {  
            e.printStackTrace();  
        }  
  
        ...  
    }  
}
```



- Maximize thread contention
 - Maximizing the number of threads running concurrently
- A cyclic barrier may be used to decrease the chance that threads are executed sequentially

```
class TestCounter {  
  
    // Shared variable for the tests  
    CyclicBarrier barrier;  
  
    ...  
  
    public void testingCounterPar  
        ...  
  
        // init barrier  
        barrier = new CyclicBarrier(nrThreads + 1);  
  
        for (int i = 0; i < nrThreads; i++) {  
            new Thread(() -> {  
                barrier.await(); // wait until all threads are ready  
                // thread execution  
                barrier.await(); // wait until all threads are finished  
            }).start();  
        }  
  
        try {  
            barrier.await();  
            barrier.await();  
        } catch (InterruptedException | BrokenBarrierException e) {  
            e.printStackTrace();  
        }  
  
        ...  
    }  
}
```

+1 is needed to wait for the main thread, i.e., the thread executing the test



- You may need to define thread classes

```
class TestCounter {  
  
    Counter count;  
    ...  
    public class Turnstile extends Thread {  
  
        private final int N;  
  
        public Turnstile(int N) { this.N = N; }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < N; i++) {  
                    count.inc();  
                }  
                barrier.await();  
            } catch (InterruptedException | BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
    ...  
}
```



- You may need to define thread classes

```
class TestCounter {  
    Counter count;  
    ...  
    public class Turnstile extends Thread {  
        private final int N;  
  
        public Turnstile(int N) { this.N = N; }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < N; i++) {  
                    count.inc();  
                }  
                barrier.await();  
            } catch (InterruptedException | BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
    ...  
}
```

Note that the thread includes
the `barrier.await()`



- You may need to define thread classes

```
class TestCounter {  
    Counter count;  
    ...  
    public class Turnstile extends Thread {  
        private final int N;  
  
        public Turnstile(int N) { this.N = N; }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < N; i++) {  
                    count.inc();  
                }  
                barrier.await();  
            } catch (InterruptedException | BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
    ...  
}
```

Note that the thread includes
the `barrier.await()`

```
class TestCounter {  
  
    // Shared variable for the tests  
    CyclicBarrier barrier;  
  
    ...  
  
    public void testingCounterParallel(int nrThreads,  
                                       int N) {  
        ...  
  
        // init barrier  
        barrier = new CyclicBarrier(nrThreads + 1);  
  
        for (int i = 0; i < nrThreads; i++) {  
            new Turnstile(N).start();  
        }  
  
        try {  
            barrier.await();  
            barrier.await();  
        } catch (InterruptedException |  
                 BrokenBarrierException e) {  
            e.printStackTrace();  
        }  
        ...  
    }  
}
```

Now we can simply start the
thread in the test



- You may need to define thread classes

```
class TestCounter {  
    Counter count;  
    ...  
    public class Turnstile extends Thread {  
        private final int N;  
  
        public Turnstile(int N) { this.N = N; }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < N; i++) {  
                    count.inc();  
                }  
                barrier.await();  
            } catch (InterruptedException | BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
    ...  
}
```

Note that the thread includes
the `barrier.await()`

```
class TestCounter {  
    // Shared variable for the tests  
    CyclicBarrier barrier;  
    private final static ExecutorService pool  
        = Executors.newCachedThreadPool();  
    ...  
    public void testingCounterParallel(int nrThreads,  
                                       int N) {  
        ...  
  
        // init barrier  
        barrier = new CyclicBarrier(nrThreads + 1);  
  
        for (int i = 0; i < nrThreads; i++) {  
            pool.execute(new Turnstile(N));  
        }  
  
        try {  
            barrier.await();  
            barrier.await();  
        } catch (InterruptedException | BrokenBarrierException e) {  
            e.printStackTrace();  
        }  
        ...  
    }  
}
```

Alternatively, we can use a
thread pool as in Goetz
We will cover ThreadPools in
lecture 9



- Optionally (though encouraged), one may generate input parameters using JUnit (@ParameterizedTest)
 - Note that the test method takes as input two integer parameters
 - Using @MethodSource we can specify a method that provides a collection of parameters (known as arguments)

```
class TestCounter {  
    ...  
    @ParameterizedTest  
    @MethodSource("argsGeneration")  
    public void testingCounterParallel(int nrThreads,  
                                       int N) {  
        //body of the test  
    }  
    ...  
}
```

```
private static List<Arguments> argsGeneration() {  
  
    // Max number of increments  
    final int I = 50_000;  
    final int iInit = 10_000;  
    final int iIncrement = 10_000;  
  
    // Max exponent number of threads (2^J)  
    final int J = 6;  
    final int jInit = 1;  
    final int jIncrement = 1;  
  
    // List to add each parameters entry  
    List<Arguments> list = new  
        ArrayList<Arguments>();  
  
    // Loop to generate each parameter entry  
    // (2^j, i) for i \in {10_000, 20_000, ..., I}  
    // and j \in {1, ..., J}  
    for (int i = iInit; i <= I; i += iIncrement) {  
        for (int j = jInit; j < J; j += jIncrement) {  
            list.add(Arguments.of((int) Math.pow(2, j), i));  
        }  
    }  
  
    // Return the list  
    return list;  
}
```



- Optionally (though encouraged), one may generate input parameters using JUnit (@ParameterizedTest)
 - Note that the test method takes as input two integer parameters
 - Using @MethodSource we can specify a method that provides a collection of parameters (known as arguments)

```
class TestCounter {  
    ...  
    @ParameterizedTest  
    @MethodSource("argsGeneration")  
    public void testingCounterParallel(int nrThreads,  
                                       int N) {  
        //body of the test  
    }  
    ...  
}
```

```
private static List<Arguments> argsGeneration() {  
  
    // Max number of increments  
    final int I = 50_000;  
    final int iInit = 10_000;  
    final int iIncrement = 10_000;  
  
    // Max exponent number of threads (2^J)  
    final int J = 6;  
    final int jInit = 1;  
    final int jIncrement = 1;  
  
    // List to add each parameters entry  
    List<Arguments> list = new  
        ArrayList<Arguments>();  
  
    // Loop to generate each parameter entry  
    // (2^j, i) for i \in {10_000, 20_000, ..., I}  
    // and j \in {1, ..., J}  
    for (int i = iInit; i <= I; i += iIncrement) {  
        for (int j = jInit; j < J; j += jIncrement) {  
            list.add(Arguments.of((int) Math.pow(2, j), i));  
        }  
    }  
  
    // Return the list  
    return list;  
}
```

Arguments is a JUnit class that can be seen as a collection of objects of different type



- Let's look at all together in code-lecture directory
- Note that Gradle requires test classes to be placed in the folder `app/src/test/java/<package>/`
- We look at three different implementations
 - CounterDR
 - CounterSync
 - CounterCAS
- JUnit produces a nice HTML report in `build/reports/tests/test/classes/<package>.<class>.html`
 - It includes outputs and running times

- Remember that some interleavings are difficult to trigger
- Let's look at the test `testingCounterParallelConstant()`
 - It is hard to find the interleavings that violate our property
 - Executing the test multiple times increases your chances of triggering the interleavings you are looking for
 - The JUnit decorator `@RepeatedTest()` can be used to run the test repeatedly

Testing a Bounded Buffer

Concurrent Correctness Test – BoundedBuffer



- Now we turn our attention to a Bounded Buffer

Consumers

Producers



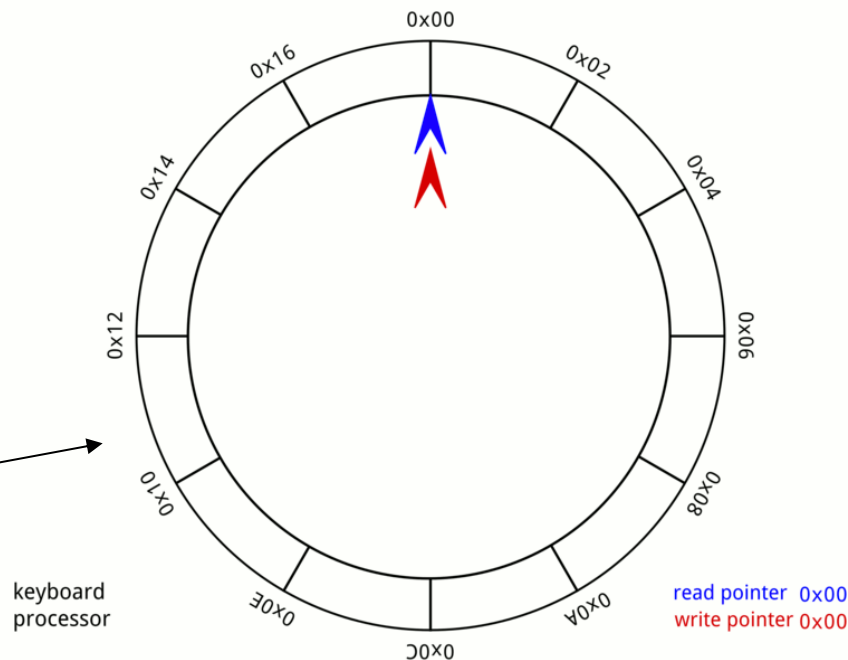
Bounded buffer

Concurrent Correctness Test – Bounded Buffer

· 41



- We study a functional correctness property of a bounded buffer that may be accessed by producers and consumers concurrently
- Producers may put elements in the buffer as long as there is space. Otherwise, they must wait
- Consumers can take elements from the buffer as long as it is not empty. Otherwise, they must wait
- The buffer is implemented as a *circular buffer*
- Synchronization is implemented using a monitor



Concurrent Correctness Test – Bounded Buffer

· 42



```
private final E[] items;
private int putPtr, takePtr, numElems;
```

```
private final Lock lock;
private final Condition notFull;
private final Condition notEmpty;
```

```
public void put(E element) {
    lock.lock();
    try {
        while(numElems >= items.length)
            notFull.await();
        doInsert(element);
        numElems++;
        notEmpty.signalAll();
        ...
    } finally { lock.unlock(); }
}
```

```
public E take() {
    lock.lock();
    try {
        while(numElems <= 0)
            notEmpty.await();
        E result = doTake();
        numElems--;
        notFull.signalAll();
        return result;
        ...
    } finally { lock.unlock(); }
}
```

- Bounded buffer implementation using a monitor
- It uses two conditions variables for threads to wait:
 - notFull – wait until buffer is not full
 - notEmpty – wait until buffer is not empty

```
private void doInsert(E element) {
    items[putPtr] = element;
    if (++putPtr == items.length) putPtr = 0;
}
```

```
private E doTake() {
    E result = items[takePtr];
    items[takePtr] = null;
    if (++takePtr == items.length) takePtr = 0;
    return result;
}
```

Concurrent Correctness Test – Bounded Buffer

· 42

```
private final E[] items;  
private int putPtr, takePtr, numElems;
```

```
private final Lock lock;  
private final Condition notFull;  
private final Condition notEmpty;
```

```
public void put(E element) {  
    lock.lock();  
    try {  
        while(numElems >= items.length)  
            notFull.await();  
        doInsert(element);  
        numElems++;  
        notEmpty.signalAll();  
        ...  
    } finally { lock.unlock(); }  
}
```

```
public E take() {  
    lock.lock();  
    try {  
        while(numElems <= 0)  
            notEmpty.await();  
        E result = doTake();  
        numElems--;  
        notFull.signalAll();  
        return result;  
        ...  
    } finally { lock.unlock(); }  
}
```

- Bounded buffer implementation using a
- It uses two conditions variables for three
 - notFull – wait until buffer is not full
 - notEmpty – wait until buffer is not empty



menti.com
2238 9090

```
private void doInsert(E element) {  
    items[putPtr] = element;  
    if (++putPtr == items.length) putPtr = 0;  
}  
  
private E doTake() {  
    E result = items[takePtr];  
    items[takePtr] = null;  
    if (++takePtr == items.length) takePtr = 0;  
    return result;  
}
```

Do we need a
lock for these
two methods?

1. Property to check

- *“after several producers put integers $x_1, \dots, x_N$ to the buffer and several consumers take integers $y_1, \dots, y_N$ from the buffer, it must hold that $\sum_{i=1}^N x_i = \sum_{i=1}^N y_i$ ”*
- More informally: *“If several threads put and take the same number of elements, the sum of the put elements and the sum of the taken elements must be equal”*
- A producer may add more than one integer in the buffer and a consumer may take more than one integer
 - *The only constraint is that the combined number of puts and takes is the same for all producers and consumers*



2. Testing setup (producer)

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Producer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Producer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                Random r = new Random();  
                for (int i = 0; i < nrTrials; i++) {  
                    int toPut = r.nextInt();  
                    buffer.put(toPut);  
                    localSum += toPut;  
                }  
                putSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (producer)

We have used two global AtomicIntegers to keep track of the global sum of put/remove

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Producer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Producer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                Random r = new Random();  
                for (int i = 0; i < nrTrials; i++) {  
                    int toPut = r.nextInt();  
                    buffer.put(toPut);  
                    localSum += toPut;  
                }  
                putSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (producer)

We have used two global AtomicIntegers to keep track of the global sum of put/remove

The producer is initialized with the number of integers it should put in the buffer. It also has a local sum of put numbers

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Producer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Producer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                Random r = new Random();  
                for (int i = 0; i < nrTrials; i++) {  
                    int toPut = r.nextInt();  
                    buffer.put(toPut);  
                    localSum += toPut;  
                }  
                putSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (producer)

We have used two global AtomicIntegers to keep track of the global sum of put/remove

The producer is initialized with the number of integers it should put in the buffer. It also has a local sum of put numbers

The producer generates a local random number to put it in the buffer. Then it updates the local sum of put numbers

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Producer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Producer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                Random r = new Random();  
                for (int i = 0; i < nrTrials; i++) {  
                    int toPut = r.nextInt();  
                    buffer.put(toPut);  
                    localSum += toPut;  
                }  
                putSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
                BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (producer)

We have used two global AtomicIntegers to keep track of the global sum of put/remove

The producer is initialized with the number of integers it should put in the buffer. It also has a local sum of put numbers

The producer generates a local random number to put it in the buffer. Then it updates the local sum of put numbers

Finally, the global put sum is updated with the local sum of the producer

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Producer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Producer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                Random r = new Random();  
                for (int i = 0; i < nrTrials; i++) {  
                    int toPut = r.nextInt();  
                    buffer.put(toPut);  
                    localSum += toPut;  
                }  
                putSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
                BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (producer)

We have used two global AtomicIntegers to keep track of the global sum of put/remove

The producer is initialized with the number of integers it should put in the buffer. It also has a local sum of put numbers

The producer generates a local random number to put it in the buffer. Then it updates the local sum of put numbers

Finally, the global put sum is updated with the local sum of the producer

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Producer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Producer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                Random r = new Random();  
                for (int i = 0; i < nrTrials; i++) {  
                    int toPut = r.nextInt();  
                    buffer.put(toPut);  
                    localSum += toPut;  
                }  
                putSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
                BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

As expected, we use a barrier to maximize contention



2. Testing setup (consumer)

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Consumer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Consumer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < nrTrials; i++) {  
                    localSum += buffer.take();  
                }  
                takeSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (consumer)

The consumer is initialized with the number of integers it should take from the buffer. It also has a local sum of taken numbers

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Consumer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Consumer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < nrTrials; i++) {  
                    localSum += buffer.take();  
                }  
                takeSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (consumer)

The consumer is initialized with the number of integers it should take from the buffer. It also has a local sum of taken numbers

The consumer takes an element from the buffer and it updates the local sum of taken integers

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Consumer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Consumer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < nrTrials; i++) {  
                    localSum += buffer.take();  
                }  
                takeSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
                BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (consumer)

The consumer is initialized with the number of integers it should take from the buffer. It also has a local sum of taken numbers

The consumer takes an element from the buffer and it updates the local sum of taken integers

Finally, the global taken sum is updated with the local sum of the consumer

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Consumer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Consumer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < nrTrials; i++) {  
                    localSum += buffer.take();  
                }  
                takeSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
                BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```



2. Testing setup (consumer)

The consumer is initialized with the number of integers it should take from the buffer. It also has a local sum of taken numbers

The consumer takes an element from the buffer and it updates the local sum of taken integers

Finally, the global taken sum is updated with the local sum of the consumer

```
Class BoundedBufferTest {  
  
    BoundedBuffer buffer;  
    AtomicInteger putSum; // global sum of put numbers  
    AtomicInteger takeSum; // global sum of taken numbers  
    ...  
    class Consumer extends Thread {  
        int nrTrials;  
        int localSum;  
  
        public Consumer(int nrTrials) {  
            this.nrTrials = nrTrials;  
            this.localSum = 0;  
        }  
  
        public void run() {  
            try {  
                barrier.await();  
                for (int i = 0; i < nrTrials; i++) {  
                    localSum += buffer.take();  
                }  
                takeSum.addAndGet(localSum);  
                barrier.await();  
            } catch (InterruptedException |  
                BrokenBarrierException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

As expected, we use a barrier to maximize contention



2. Testing setup (test)

```
public void putTakeTest(int nrThreads,
                        int nrTrials,
                        int bufferSize) {

    // init buffer
    buffer = new BoundedBufferMonitor<Integer>(bufferSize);
    // init barrier
    barrier = new CyclicBarrier((nrThreads*2) + 1);

    for (int i = 0; i < nrThreads; i++) {
        new Producer(nrTrials).start();
        new Consumer(nrTrials).start();
    }

    try {
        barrier.await();
        barrier.await();
    } catch (InterruptedException | BrokenBarrierException e)
    {
        e.printStackTrace();
    }
    assert(putSum.get() == takeSum.get());
}
```



2. Testing setup (test)

The test has 3 parameters: the number of pairs of producer consumers, the number of put/take that each producer/consumer must perform and the size of the buffer

```
public void putTakeTest(int nrThreads,
                        int nrTrials,
                        int bufferSize) {

    // init buffer
    buffer = new BoundedBufferMonitor<Integer>(bufferSize);
    // init barrier
    barrier = new CyclicBarrier((nrThreads*2) + 1);

    for (int i = 0; i < nrThreads; i++) {
        new Producer(nrTrials).start();
        new Consumer(nrTrials).start();
    }

    try {
        barrier.await();
        barrier.await();
    } catch (InterruptedException | BrokenBarrierException e)
    {
        e.printStackTrace();
    }
    assert(putSum.get() == takeSum.get());
}
```



2. Testing setup (test)

The test has 3 parameters: the number of pairs of producer consumers, the number of put/take that each producer/consumer must perform and the size of the buffer

We initialize the buffer and the barrier

```
public void putTakeTest(int nrThreads,
                        int nrTrials,
                        int bufferSize) {

    // init buffer
    buffer = new BoundedBufferMonitor<Integer>(bufferSize);
    // init barrier
    barrier = new CyclicBarrier((nrThreads*2) + 1);

    for (int i = 0; i < nrThreads; i++) {
        new Producer(nrTrials).start();
        new Consumer(nrTrials).start();
    }

    try {
        barrier.await();
        barrier.await();
    } catch (InterruptedException | BrokenBarrierException e) {
        {
            e.printStackTrace();
        }
    }
    assert(putSum.get() == takeSum.get());
}
```



2. Testing setup (test)

The test has 3 parameters: the number of pairs of producer consumers, the number of put/take that each producer/consumer must perform and the size of the buffer

We initialize the buffer and the barrier

```
public void putTakeTest(int nrThreads,
                        int nrTrials,
                        int bufferSize) {

    // init buffer
    buffer = new BoundedBufferMonitor<Integer>(bufferSize);
    // init barrier
    barrier = new CyclicBarrier((nrThreads*2) + 1);

    for (int i = 0; i < nrThreads; i++) {
        new Producer(nrTrials).start();
        new Consumer(nrTrials).start();
    }

    try {
        barrier.await();
        barrier.await();
    } catch (InterruptedException | BrokenBarrierException e) {
        {
            e.printStackTrace();
        }
    }
    assert(putSum.get() == takeSum.get());
}
```

We execute a producer and consumer in each iteration



2. Testing setup (test)

The test has 3 parameters: the number of pairs of producer consumers, the number of put/take that each producer/consumer must perform and the size of the buffer

We initialize the buffer and the barrier

Recall: the first await is to maximize contention, and the second to wait for all threads to finish execution

```
public void putTakeTest(int nrThreads,
                        int nrTrials,
                        int bufferSize) {

    // init buffer
    buffer = new BoundedBufferMonitor<Integer>(bufferSize);
    // init barrier
    barrier = new CyclicBarrier((nrThreads*2) + 1);

    for (int i = 0; i < nrThreads; i++) {
        new Producer(nrTrials).start();
        new Consumer(nrTrials).start();
    }

    try {
        barrier.await();
        barrier.await();
    } catch (InterruptedException | BrokenBarrierException e) {
        e.printStackTrace();
    }
    assert(putSum.get() == takeSum.get());
}
```

We execute a producer and consumer in each iteration



2. Testing setup (test)

The test has 3 parameters: the number of pairs of producer consumers, the number of put/take that each producer/consumer must perform and the size of the buffer

We initialize the buffer and the barrier

Recall: the first await is to maximize contention, and the second to wait for all threads to finish execution

Finally, we check that our property holds after executing the test. The test relies on the correctness of AtomicInteger

```
public void putTakeTest(int nrThreads,
                        int nrTrials,
                        int bufferSize) {

    // init buffer
    buffer = new BoundedBufferMonitor<Integer>(bufferSize);
    // init barrier
    barrier = new CyclicBarrier((nrThreads*2) + 1);

    for (int i = 0; i < nrThreads; i++) {
        new Producer(nrTrials).start();
        new Consumer(nrTrials).start();
    }

    try {
        barrier.await();
        barrier.await();
    } catch (InterruptedException | BrokenBarrierException e) {
        e.printStackTrace();
    }
    assert(putSum.get() == takeSum.get());
}
```

We execute a producer and consumer in each iteration



2. Testing setup (test)

The test has 3 parameters: the number of pairs of producer consumers, the number of put/take that each producer/consumer must perform and the size of the buffer

We initialize the buffer and the barrier

Recall: the first await is to maximize contention, and the second to wait for all threads to finish execution

Finally, we check that our property holds after executing the test. The test relies on the correctness of AtomicInteger

```
public void putTakeTest(int nrThreads,
                        int nrTrials,
                        int bufferSize) {

    // init buffer
    buffer = new BoundedBufferMonitor<Integer>(bufferSize);
    // init barrier
    barrier = new CyclicBarrier((nrThreads*2) + 1);

    for (int i = 0; i < nrThreads; i++) {
        new Producer(nrTrials).start();
        new Consumer(nrTrials).start();
    }

    try {
        barrier.await();
        barrier.await();
    } catch (InterruptedException | BrokenBarrierException e) {
        e.printStackTrace();
    }
    assert(putSum.get() == takeSum.get());
}
```

We execute a producer and consumer in each iteration

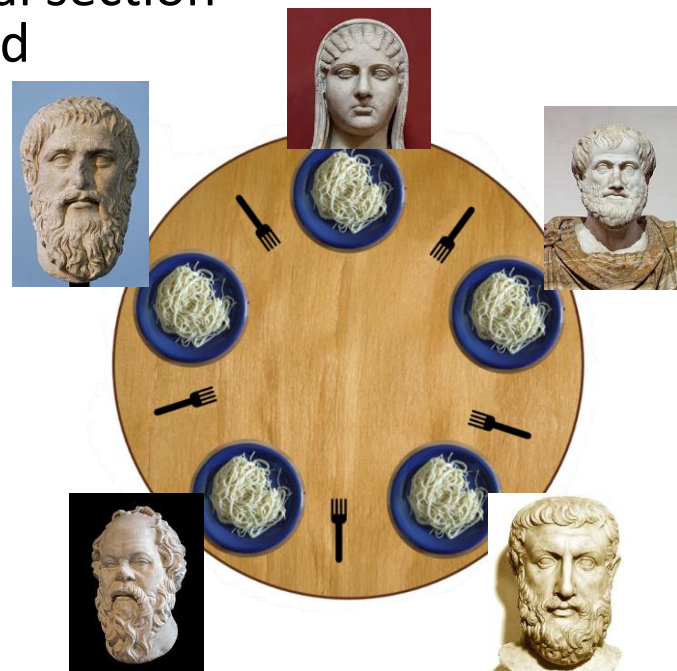
Let's run the test!



- The JDK has a built-in test suite for concurrent programs
 - [Java Concurrency Stress](#) (jcstress)
 - The lecture code for this week contains a gradle jcstress plugin and an example test class (see `week05lecture/README.md`)
- The testing suite [Coyote](#) for C# uses a *smart* scheduler that helps explore different interleavings in each test execution



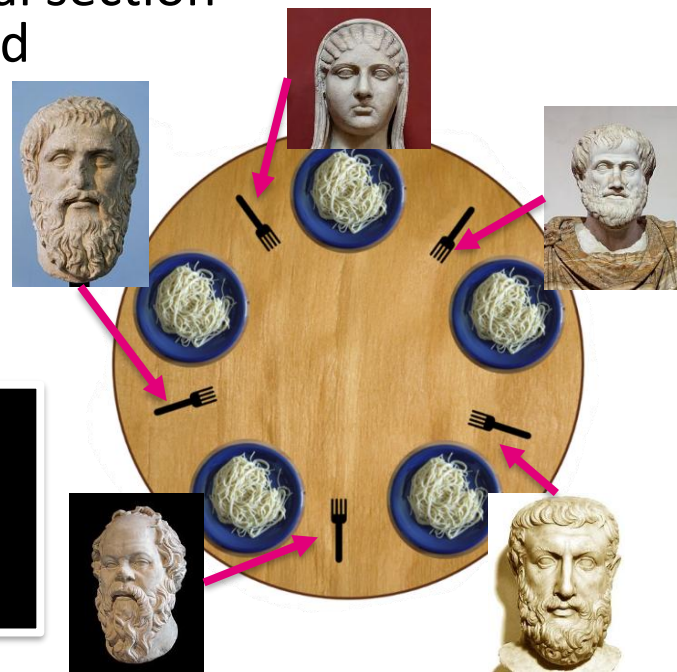
- A deadlock occurs when a thread does not eventually leave the critical section; thus it prevents other threads from executing the critical section
 - For instance, if all threads are in a critical section waiting for a lock held by another thread
- Standard example:
 - Dining philosophers by E.W. Dijkstra
 - Philosophers only think and eat
 - A philosopher must pick both left and right forks to start eating





- A deadlock occurs when a thread does not eventually leave the critical section; thus it prevents other threads from executing the critical section
 - For instance, if all threads are in a critical section waiting for a lock held by another thread
- Standard example:
 - Dining philosophers by E.W. Dijkstra
 - Philosophers only think and eat
 - A philosopher must pick both

If all philosophers grab their right fork, we reach a deadlock state. Note that this behaviour is captured by a *finite* interleaving.



Testing deadlocks



- Testing for deadlocks is complicated and often not possible

- Testing for deadlocks is complicated and often not possible

Is deadlock-freedom a safety property or a liveness property? Why?



menti.com
2238 9090

- Writing tests to find deadlocks is complicated
- We must be able to characterize the state that leads to the deadlock using assertions (e.g., all philosophers have grabbed their right fork)
- When running a test, if we observe that the program does not terminate for a long time, it might be due to deadlocks (but not necessarily)
 - Let's run the previous test on an implementation of a bounded buffer with deadlocks

Formal Verification

- Testing is an extremely useful technique, which is the de-facto approach in industry
 - You should extensively test all your programs!
- However, it cannot be used to prove the absence bugs (remember the first slides)
- Tests can be seen as interleaving generators 😊
 - They execute the system to produce different interleavings
 - For most systems, it is virtually impossible to write a set of tests that cover all possible interleavings in the system



- Formal verification is a technology that aims to *prove* that a program satisfy a specification (properties)
- It treats programs and properties as mathematical objects
- Using mathematical reasoning it is possible to prove that programs satisfy their specifications (i.e., for all possible interleavings)
 - Manually: Proof assistants (Rocq, Isabelle, Lean, etc.)
 - Automatically: SAT solvers, SMT solvers, **model-checking**, static verification, symbolic execution, etc.

- Model-checking transforms programs into a *finite-state* models that encapsulate all possible interleavings in the system
 - Automata, Kripke structures, binary decision diagrams, etc.
- Properties are specified in some type of logic
 - Linear Temporal Logic (LTL), Computational Tree Logic (CTL), First-Order Logic (FOL), propositional logic, etc.
- The model of the program and the property are typically expressed in the same language, so it is possible to automatically check whether they are satisfied
- Model-checking has been very successful in hardware verification at Intel

JavaPathFinder (switch to rocket science)



- JavaPathFinder is (among other things) a model-checker for Java programs
- It is developed at the Jet Propulsion Lab (JPL) at NASA
 - It was used to verify part of the system in the Curiosity rover that landed in Mars in 2012
- Let's look at a few examples of using JavaPathFinder



Rocket science!



Threads in Java – Example II

· 63



- Altogether (not executable) the executable program

HARDer: What is the minimum value of **counter** that this program can print?



```
long counter = 0;  
final long PEOPLE = 10_000;
```

```
Turnstile turnstile1 = new Turnstile();  
Turnstile turnstile2 = new Turnstile();  
turnstile1.start();turnstile2.start();  
turnstile2.join();turnstile2.join();  
System.out.println(counter+" people entered");
```

```
public class Turnstile extends Thread {  
    public void run() {  
        for (int i = 0; i < PEOPLE; i++) {  
            counter++;  
        }  
    }  
}
```

Another déjà vu

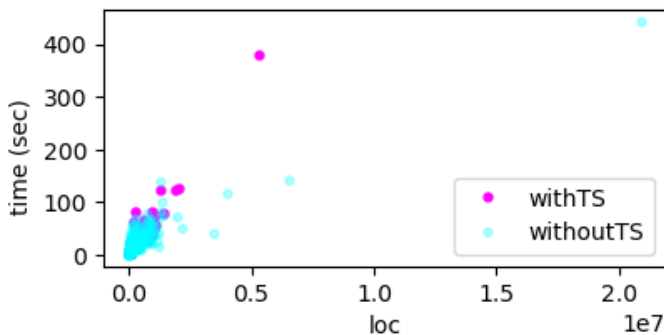
Let's use javapathfinder to automatically get the answer

- Let's now look at an example of finding deadlocks with JavaPathFinder in a buggy implementation of a bounded buffer

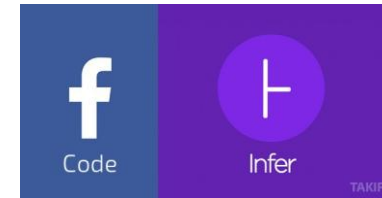
Too good to be true...



- If formal verification is so good, why isn't everyone using it all the time?
 - Welcome to the **state explosion problem**! (among other things)
 - Even for small programs the computational cost of proving that the system satisfies its specification can be astronomically expensive
- **The use of abstractions, approximations or narrowing down the problem domain has helped formal verification to scale better**
 - Example (part of my research):
Finding data races in Java
classes in large code bases using
CodeQL (static analysis)
(less than 2min to analyze 200k LOC)
(optional readings from last week)



- Many companies have started to use formal verification in their software development process, so it might be a good asset to have in your toolbox



- At ITU, you can learn more about formal verification in the *software analysis specialization*
- I believe software engineers should be aware of this technology and trained to use it (warning: personal opinion)
- LLM-assisted software development makes the need for this type of program analysis methods more important than ever

- Formal verification is an active topic of research
- ***If you found this topic interesting, feel free to contact me regarding MSc thesis projects***
 - Also keep an eye on people working at the Software Quality Group (SQUARE), the Centre of Security and Trust (CISAT) and the Programming, Logic and Semantics (PLS) group
- My interests focus on using formal verification to
 - Prove that systems satisfy legal privacy requirements (e.g., GDPR)
 - Quantify privacy risks in machine learning
 - Prove correctness properties in probabilistic programs

- Intro to concurrency properties
- Testing
 - Intro to JUnit
 - Counter
 - Bounded Buffer
 - Deadlocks
- BONUS: Formal Verification
 - Java Path Finder

